Proceedings of the International Conference on Inventive Computation Technologies (ICICT 2023)
IEEE Xplore Part Number: CFP23F70-ART; ISBN: 979-8-3503-9849-6

Word Segmentation and Sandhi Resolution on Ayurveda Classical Scriptures

Amrita Varshini E R
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Amritapuri, India
amritavarshinier@am.students.amrita.edu

Jayashree Nair
Department of Computer Science and Applications
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Amritapuri, India
jayashree@am.amrita.edu

2023 International Conference on Inventive Computation Technologies (ICICT) | 979-8-3503-9849-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICICT57646.2023.10134024

***Abstract*—Sanskrit is one of the oldest of the Indo-Aryan languages which has an abundant amount of texts in literature. One of the main sources of these Sanskrit texts is the textbooks written about the field of Ayurveda, the ancient Indian medical system. However, proper computational systems for processing this language are bare. A very challenging part of processing the Sanskrit language is the task of *Sandhi* resolution, which means the splitting of phonetically merged words. This study mainly focuses on this complex task of joint compound splitting and *Sandhi* resolution in Sanskrit specific to Ayurveda texts. This paper also surveys and compares all the existing research on *Sandhi* resolution in Sanskrit. In this work, an Ayurvedic dataset is compiled from some popular Ayurvedic texts, and a performance analysis of the state-of-the-art word segmentation model is carried out using this dataset.**

***Index Terms*—Sanskrit, Word Segmentation, *Sandhi* Resolution, Ayurveda**

I. Introduction

Natural Language Processing (NLP) systems have seen a widespread level of significance lately. More and more text and speech-related tasks getting automated are resulting in ease and better life quality for humans considering many daily activities. However, a point of concern remains about how these go true only for languages with abundant resources and tools available. Computationally challenging languages still face the barrier of having insufficient NLP systems for their applications. Sanskrit, though an ancient Indian language, is still one such language. The first texts in this language date back to around 1500 B.C [1]. With the advent of the digitization of Sanskrit scriptures, a lot of research attempt is being done on various downstream NLP tasks in Sanskrit [2]. However, the current works are on general Sanskrit literature and scriptures only.

The medical field of Ayurveda is a vast field of knowledge that has its roots in Sanskrit and Indian traditions. The *Atharva Veda* is credited with giving rise to Ayurveda because it lists numerous ailments and their cures [3]. Later, from the 6th century BC to the 7th century AD, there was systematic growth in this field of knowledge. During this period, a wide range of classical Ayurvedic works were written by several scholars, practised and systemized in the entire country of India. This period called the *Samhita* period, is hence considered to be the golden age of Ayurveda [4]. There is an abundant volume of unstructured textual data related to Ayurveda, available as numerous texts describing various ayurvedic substances, medicines, and techniques [5]. Yet, these are still far away from computational processing due to the linguistic challenges in Sanskrit [6]. These challenges are found in all linguistic levels be its phonetic, morphology, syntax, or semantics. Thus, the research focus on Ayurvedic scriptures remains open. It could be utilized as a major source of data when it comes to the language of Sanskrit. Although we can observe quite some works focused on Natural Language Processing tasks when it comes to Sanskrit language texts, works on Ayurvedic texts in the Sanskrit language are limited. Also, when coming to Ayurvedic texts, the sentences are much lengthier and poetic in nature. Furthermore, there are compounds which could be medical terms which would prefer to be left as it is rather than splitting them into their constituent words.

In Sanskrit, *Sandhi* is the phonetic combination that arises during the merger of two words into one. Here, the phonemes present at the boundaries of the merger transform into another or one of the same phonemes based on some predefined euphonical rules [7]. Across word borders, euphonic alterations take place, adding and removing parts of the merging words' original components [8]. The segmentation in Sanskrit is made less certain by these euphonic modifications [9]. As an example of *Sandhi*, consider the Sanskrit string *rājo̲vaca* ("the king said") which has one *Sandhi*. The phoneme *o̲* is created by the combination of *ā* (the final letter of *rājā*) and *u* (the initial letter of *uvāca*). When splitting the compound, we'll get *rājā uvāca*. A similar process can also be observed in other languages [10]. For example, in English, '*come*' + '*ing*' → '*coming*', where we lose the additional '*e*' in the word '*come*' while merging [11]. In Sanskrit, such compound words can be split into various ways [12]. The semantically appropriate split of words that applies to the given word can only be identified by the context in which it appears. Hence, the task of splitting these compound words requires knowledge of the context in which the compound exists. This task becomes much more complex when it comes to Ayurvedic scriptures which would require different methods of handling these splits. In this study, we aim to analyse this challenge of *sandhi* resolution in Ayurvedic classical scriptures. As an initial stage, a review of some of the state-of-the-art *sandhi*

resolution models was conducted. Then the focus was on the compilation of some digitally available popular ayurvedic texts into a dataset/corpus. One of the reviewed models was applied to this dataset to get the *sandhi*-resolved version of the dataset. Other reviewed models were also attempted to be tested on the dataset which couldn't be completed due to unresolvable errors in the available resources of the models.

The rest of the paper is organized as follows. Section II briefs the review conducted of some of the existing word segmentation models. Section III details the methodology and framework followed for the conduction of the study. In Section IV, a discussion on the tools, techniques, experimental environment, dataset and evaluation metrics is given along with the obtained results. Section V describes the contributions and limitations drawn out from the study. Finally, Section VI concludes the research and provides a direction for further research work.

II. LITERATURE REVIEW

Hellwig approaches the task of *Sandhi* resolution in the Sanskrit language in another way by building a classifier [1] which relies entirely on gold standard string splits, without any other external resources. The data for training is obtained from Hellwig's SanskritTagger [13] corpus where the sentences and re-analyzed and produced such that it matches its gold standard in the database. The dataset is produced such that there is a source and target sequence, where the former is the string split into phonemes, and the latter is the sequence with transformations involved for each of the phonemes. The classifier is made to learn how to accurately split a compound given with appropriate *sandhi* resolution if exists and to generate the associated *sandhi* rule which got applied in the process. 5 possible transformations (R_{1-5}) are listed based on which the classification of phonemes in each string occurs. The classifier shows better results in labelling the R_1(unchanged phoneme) and R_5(applying *sandhi* with dependency on the initial phoneme of the next string). The other labels, which are more complicated to learn showed a lower F score in the range of 90-93. The string-wise accuracy of the classifier was found to be getting better when trained on more data. Longer strings with 6-15 phonemes showcased more misclassifications where any one phoneme is misclassified. The proposed algorithm could be applied to other less-resourced languages such as Old Marathi or premodern Nepali for primary analyses.

The biggest challenge in the task of Sanskrit word segmentation is to identify the most semantically accurate segmentation among the multiple possible splits which could be derived from a compound word. This has been addressed by Krishna et al. [12] by employing word segmentation as a query expansion problem utilizing path constrained random walks (PCRW) framework. The Digital Corpus of Sanskrit (DCS) is used in this work which contains over 430,000 segmented sentences and over 3.2 million segmented tokens. From the DCS corpus, 100,000 sentences were used in this work with 10,000 sentences used as held-out data which was used only for testing the framework in the end. Sanskrit Heritage Reader [14]–[16] is used as a morphological analyzer to produce the candidate segments and associated details of an input sequence. Using these candidate segments as nodes, a graph is formulated. PCRW with the longest sequence as the starting node is carried out to obtain a winner node to be added to the context and eliminate all the opposing nodes to it. This is repeated until no more candidate segments are left to be assessed. In addition to PCRW, the authors incorporated morphological details using Inductive Logic Programming (ILP) which showed quite a good hike in performance. The framework showcases an impressive score of precision, recall and F1-score in comparison with the baseline models. When applied to the held-out dataset, the model showed a slightly deprecated performance of nearly 3% F-score compared to the results obtained on the original dataset. Also, the model was found to be working similarly for sentences of varying lengths. Also, compared to the best-performing baseline system, the proposed framework was successful in predicting all correct splits by a margin of 19% of all the sentences.

Tokenization of Sanskrit involves jointly splitting compound words along with the required *Sandhi* resolution. This depends on the local phonetic and contextual features of the given textual data. Hellwig et al. [17] attempt to integrate these properties while tokenizing Sanskrit by implementing convolutional and recurrent elements interchangeably to their model and by even adding shortcut connections. Three models have been introduced by the authors to achieve the goal of tokenization: $crNN$(convolutional elements followed by bidirectional recurrent layer), $rcNN$ (switches the order of convolutional and recurrent elements in $crNN$), and $rcNN_{short}$ (an extension of $rcNN$ where shortcut connections are added). Among the three models which were implemented, the best performing one was $rcNN_{short}$ with split probabilities incorporated into it. Further, the models are trained with and without split probabilities and the string accuracy is calculated using the McNemar test. The authors point out the scope of expansion of the model into the joint learning of splits, and lexical and morphological annotations to improve the results and suggest the idea of using modified segmental NNs for this purpose. A new dataset was also put forth, derived by re-analysing 560,000 sentences from the Digital Corpus of Sanskrit (DCS). This was done using SanskritTagger software proposed by Hellwig in one of his previous works [13].

Aralikatte et al. puts forth Double Decoder Recurrent Neural Network (DD-RNN) [11], an attention-based deep learning architecture aiming to predict the split location in each compound and subsequently identify the *sandhi* resolved words of the compound. Predicting the split location before predicting just the *sandhi* rules, as seen in other models, is to reduce the number of trials of splits to be applied on the compounds. The model is also shown to be positively applicable to the task of Chinese word segmentation. The authors compiled a benchmark dataset of 71,474 words by extracting and combining standard splits from multiple UoH corpus and *SandhiKosh* corpus [18]. In the DD-RNN model (seq2(seq)2), a bidirectional encoder initially takes in the compound word

character-by-character and this is used by the global attention layer which is expected to improve the identification of split location. There are two decoders, a location decoder to predict the location of the split and a character decoder to learn the *sandhi* split rules. Both decoders are trained separately with one of them being frozen at a time. Comparison of the DD-RNN model with publicly available *sandhi*-splitting tools showed its out-performance by a margin of at least 20% in terms of accuracy. Comparison with other RNN architectures also showed improved accuracy in DD-RNN with location prediction being 95% accurate and split prediction having a score of 79.5%. When coming to Chinese word segmentation models [19], [20] too, DD-RNN outperformed them in split location prediction by a margin of over 25%.

Reddy et al. [5] attempt to tackle this complex task of word segmentation in the Sanskrit language using a deep sequence to sequence (seq2seq) model using LSTMs at both encoder and decoder sides to predict un-*sandhi*ed sentences given a *sandhi*ed sentence as input. The model focuses solely on the identification of word splits and the correctness of un-*sandhi*ed strings from a *sandhi*ed input string independent of any morphological or semantic details. The proposed model with and without attention module was compared to the baseline models of supervisedPCRW and GraphCRF using the string-wise micro-averaged precision, recall and F-score. The incorporation of the attention module was a successful attempt as it showcased better performance by a margin of 16.29% in the F-score from the baseline models. An instance where the baseline model outperformed the attention model is when the sentence length is more than 10 words.

A machine learning approach for the classification of compound words, when given a Sanskrit text, was taken by Premjith et al. [21] by using many classification methods among which the K-Nearest Neighbor method performed the best. In order to capture any minuscule morphological information concealed in the Sanskrit words, fastText word embedding was utilised to vectorize the Sanskrit words and the entire task was modelled as a binary classification problem. Naive Bayes, K-Nearest Neighbor, Decision Tree, Random Forest, Support Vector Machine, Multi-Layer Perceptron, Logistic Regression, and Adaboost classifier are some of the machine learning techniques that were employed for the challenge. The authors used the UoH (University of Hyderabad) tagged dataset available online which has 32,183 tokens including decomposed compound words as well as undecomposed non-compound words. An important observation made was the non-linearity in the data which resulted in methods such as Support Vector Machines underperforming.

Krishna et al. proposed a generic graph-based structure prediction framework (SPF) using Energy-Based Models (EBM) for various syntactic level NLP tasks in Sanskrit [7]. The syntactic tasks include word segmentation, morphological parsing, and linearization. In an SPF the input is a graph and the output is a subgraph of the input. Here the input graph is generated for an input text using the Sanskrit Heritage Reader (SHR) [14]–[16]. An EBM ML model is trained to find the best subgraph with minimum energy. The proposed system, Cliq-EBM-F, reports the best score in terms of macro average Precision, Recall, and F-Score for the task. It reports a percentage improvement of 1.15% over baseline work, rcNN-SS, in F-Score. It was trained on 8,200 sentences, which is just about 1.5% of the training data (¿ 0:55 million) used in rcNN-SS.

Leander Girrbach and Jingwen Li detail the approach taken by them in their submission [22] to the Word Segmentation and Morphological Parsing (WSMP) for the Sanskrit hackathon. The approach taken was that of sequence labelling where the morphological tags and rules were to be predicted. There were three tasks for which the solution was proposed: word segmentation, morphological parsing and combined segmentation and analysis. Task 1 model showed a score of nearly 96 for F1, precision and recall. For task 2, the evaluation showed the full match of sentences to be around 65%, correct word prediction to be around 92% and around 87% of erroneous predictions to be partially correct.

III. METHODOLOGY

The initial step in the proposed methodology of this research, shown in Fig. 1, was to consolidate the existing works related to *sandhi* resolution and word segmentation in the Sanskrit language. The next important and vital requirement for this study was the curation of a dataset compiling different ayurvedic classical scriptures. This task of dataset compilation had different phases which included the listing of available ayurvedic scriptures, using appropriate tools and techniques to acquire them and preprocessing them to make them compatible with the models to be experimented with.

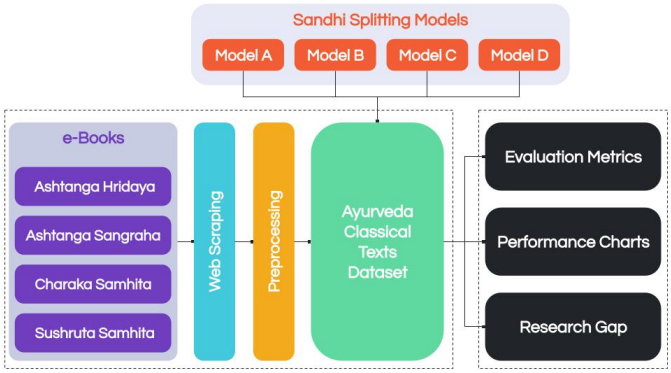

Fig. 1. System Design Diagram

First, the available Ayurvedic textbooks were listed. Later, the required tools and techniques to acquire these textbooks into a corpus were researched and experimented with depending on the format in which these textbooks were available. Once we had proper channels of acquisition of data, the data was compiled into a single corpus where each document contained one Ayurveda textbook. The textbooks used for the corpus compilation were *Ashtanga Hridaya*, *Ashtanga Sangraha*, *Charaka Samhita* and *Sushruta Samhita*. All four of these were available as ebooks which were extracted using

web scraping tools. Once the dataset was compiled, the chosen word segmentation models were to be applied to the dataset. For this application, it was required to process the dataset into specific formats to make it compatible with the models in hand. This preprocessing mainly comprised transliteration tasks and changing the transliteration schemes according to the models. This was carried out using the devatrans tool which enables transliteration tasks specifically for the Sanskrit language.

A. Model A: Character-level Recurrent and Convolutional Neural Networks

Hellwig put forth three models using character-level recurrent and convolutional neural networks [17]: $crNN$ (convolutional elements followed by bidirectional recurrent layer), $rcNN$ (switches the order of convolutional and recurrent elements in $crNN$), and $rcNN_{short}$ (an extension of $rcNN$ where shortcut connections [23] are added). Among the three models, $rcNN$ showcased the best performance while $crNN$ showed equivalent performance to that of the baseline models. Also, the errors in $rcNN_{short}$ turn out semantically meaningful much more frequently whereas those of $crNN$ are real mis-segmentations showing its inability to incorporate semantic influence. Hence, this $rcNN_{short}$ was taken to be tested on the Ayurvedic dataset.

B. Model B: Sandhi Vicceda Model

Dave et al. proposed a *Sandhi* generation and splitting model [24] using sequence to sequence model claiming that the *Sandhi* task is similar to language translation tasks which demand such models. In the *sandhi* splitting model which was put forth, the authors staged the task into two, the first being the prediction of the *sandhi*-window and the second being the splitting of the predicted *sandhi*-window. In the *sandhi*-window prediction, an integer array corresponding to the input sequence is the expected output where only the elements corresponding to the characters in the *sandhi*-window are set as 1 and all others are set as 0. This window is then split into 2 using a seq2seq model which uses an LSTM as basic RNN cell as the decoder and Bi-LSTM as basic RNN for the encoder with a hidden unit size of 128.

C. Model C: seq2seq model

Reddy et al. derived this model [5] from the neural machine translation model of Wu et al. [25] and incorporated the idea of input string reversal from Sutskever et al. [26] due to the improved performance. The model has 3 deep layers at the encoder and decoder end to get a better-performing LSTM unit. Trials with and without attention modules were also carried out. The incorporation of the attention module was a successful attempt as it showcased better performance by a margin of 16.29% in the F-score from the baseline models. A completely opposite scenario is noticed in the no-attention model where it has a deprecated performance of 6.29% in the F-score. Upon experimenting with the attention-module incorporated model on our Ayurvedic dataset, the outdated version of the packages in the code posed a roadblock for the testing. The code was available in python 2, which was manually converted to python 3. Later on, the tensorflow version used was upgraded using the *TF1* to *TF2* migration command-line script *tf_upgrade_v2*. However, some of the functionalities such as *rnn_cell* and *seq2seq* were not upgraded due to major shifts of packages in the current versions. This caused a barrier in moving forward with the testing.

Currently, the model put forth by Hellwig [17] has been tested on the ayurvedic dataset. In the case of Model B, the required transliteration of the dataset to the SLP1 scheme has been carried out and the model has been tried out on the dataset. However, the input format compatibility issues have been causing difficulties in moving forward with testing. In the case of Model C, the model uses outdated versions of packages tensorflow and its related dependencies which have been found to be much more time-consuming to be resolved. Multiple attempts at trying various versions and degradation of the code were carried out, however, the efforts were proved futile.

IV. RESULTS AND DISCUSSION

A. Tools, Techniques and Experiment Environment

1) Selenium: Selenium is a set of tools and libraries which allows web browser automation. It is available as a python library which has been made use of in this project to carry out the web scraping required to extract the ayurvedic textbooks which were available as ebooks on different online platforms.

2) devatrans: [1]devatrans is a transliteration tool specifically for the Sanskrit language. This tool uses a rule-based system and supports transliteration, back transliteration [27], and inter transliteration in the following schemes: International Alphabet of Sanskrit Transliteration(IAST), Indian language transliteration(ITRANS), Harvard-Kyoto(HK) and Velthuis. This tool was used in the processing of the ayurvedic dataset to accommodate the specific models which were taken for testing on the said dataset.

B. Dataset

The ayurvedic dataset was compiled from four renowned Ayurveda classical scriptures, namely [2]Ashtanga Hridaya, [3]Ashtanga Sangraha, [4]Charaka Samhita and [5]Sushruta Samhita. These scriptures are composed of *sthānas* (books) each explaining specific topics and each of these *sthānas* is further divided into 100+ *adhyāyas* (chapters). These textbooks are considered the most principal texts of Ayurveda which explain many of the ancient medical treatises and all of them discuss many similar subjects such as General Principles, Pathology, Diagnosis, Anatomy, Sensorial Prognosis, Therapeutics, Pharmaceutics and Toxicology.

[1]https://pypi.org/project/devatrans/
[2]https://ayurvidya.net/shastra/hridayam/
[3]https://ayurvidya.net/shastra/sangraha/
[4]https://niimh.nic.in/ebooks/ecaraka/
[5]https://niimh.nic.in/ebooks/esushruta/

TABLE I
AYURVEDIC DATASET DESCRIPTION

Ayurveda Text	*Sthānas*	Chapters	Lines scraped
Ashtanga Hridaya	6	120	10775
Ashtanga Sangraha	6	150	6731
Charaka Samhita	8	120	10268
Sushruta Samhita	6	186	9670

Each of these textbooks was scraped into individual documents with a total of over 35,000 lines scraped. These were also transliterated into IAST and ITRANS using devatrans python library for further applications. Information regarding *sthānas*, chapters and the number of scraped lines in each scripture has been detailed in Table 1.

C. Evaluation Metrics

The testing of reviewed models on the Ayurvedic dataset is to be evaluated on the basis of the standard metrics of evaluations when it comes to word segmentation models, namely precision, recall, accuracy and F1-score.

1) Accuracy: Accuracy simply calculates the ratio of the correct predictions to the total number of observations. However, it is not completely reliable unless the dataset at hand is equally distributed.

2) Precision: Precision, also known as a positive predictive value, defines the exactness or quality of the prediction. It is measured by taking the fraction of the number of correctly predicted positive labels and the total number of labels that have been predicted as positive.

3) Recall: The recall is also referred to as sensitivity and it measures the ability of the model to identify the actual positive labels out of all the positive labels existing in the dataset.

4) F1-score: F1-score is considered a much more reliable score than accuracy when the dataset is not symmetric. It basically takes a weighted average of the precision score and recall score.

D. Experiments and Results

Among the various word segmentation models which have been reviewed, the best-performing model from Hellwig's work which uses character-level recurrent and convolutional neural networks [17], Model A was applied to the compiled Ayurvedic dataset. This was the $rcNN_{short}$ model which uses shortcut connections and split probabilities. Model A showcased an impressive performance of 96.7 accuracy and 94.8 string-based recall on the model's original dataset. The model's evaluation on real-world data was done in the original work by testing it on some sentences from the Buddhist treatise *Trimsikavijnaptibhasya* and the philosophical text *Nyāyamañjarī*. This showed a deprecated value in the performance metrics with a drop of around 20-30. This was expected due to the mismatched conventions of these texts when considering the training data. The pre-trained model was available which was applicable to un-*sandhi*ed text files which in turn generates its corresponding *sandhi*ed text file as output. The model expects the input text file content to be Sanskrit language text in the IAST transliteration schema encoded in UTF-8. We used devatrans library to process our dataset into this schema to accommodate this model. Since the model was trained on classical Sanskrit datasets, it is expected to perform satisfactorily on our Ayurvedic dataset too. Detailed evaluation of the output generated using this model on the Ayurvedic dataset requires manual assistance as no prior data related to the *sandhi*ed versions of these textbooks are available. Some notable observations are how the model cannot handle over 128 characters in a single line. We can observe many of the lines in the *sandhi*ed texts being omitted after the 128th character. Some sample inputs and outputs have been shown in Fig. 2 and Fig. 3. In order to arrive at a proper result of the experiment, manual correction of the obtained outcomes has to be conducted from which the evaluation metrics have to be calculated and compared with the original ones.

Further models with available resources which could be tested on our compiled Ayurveda dataset are the *Sandhi Vicceda* model proposed by Dave et al. (Model B) [22] and the seq2seq model put forth by Reddy et al. (Model C) [5]. Model B is based on a seq2seq model which firsts predict the *sandhi*-window and then splits the predicted window, both using sequential models. Model C was trained on a dataset of 107,000 sentences taken from the Digital Corpus of Sanskrit.

V. CONTRIBUTIONS AND LIMITATIONS

The study of the word segmentation models and their performance on the ayurveda dataset helps in drawing a summary of the drawbacks that could be resolved on the existing models to accommodate Ayurvedic text datasets and the possible directions of future research to be taken for the computational availability of systems to support the processing of Ayurveda texts.

Among our models chosen for testing, Model A shows a major drawback of not supporting text lines of over 128 characters in length resulting in the complete omission of a huge part of our data. This could be corrected by training the model from scratch using our dataset so that it learns to support longer and more complex lines of text. Model B has shown limitations as it required target outputs in order to carry out the testing of the model which is not available with our dataset. For Model C, a new vocabulary file would be required to run the model since the Ayurveda dataset includes many medical terminologies which may need to be processed differently unlike the normal Sanskrit language. Upon trials to carry out these said procedures, version compatibility issues and outdated version of the available code of the model has posed a barrier to continuing the testing.

The need for manual assistance in order to evaluate the output un*sandhi*ed texts is a very challenging task. This has led to the decision of evaluating a few selected lines of outputs from the tested dataset manually and deriving a conclusion. The dataset could be expanded much more with other available

```
dīrghaṃ jīvitamanvicchanbharadvāja upāgamat| indramugratapā buddhvā śaraṇyamamareśvaram||3||
brahmaṇā hi yathāproktamāyurvedaṃ prajāpatiḥ| jagrāha nikhilenādāvaśvinau tu punastataḥ||4||
aśvibhyāṃ bhagavāñchakraḥ pratipede ha kevalam| ṛṣiprokto bharadvājastasmācchakramupāgamat||5||
vighnabhūtā yadā rogāḥ prādurbhūtāḥ śarīriṇām| tapopavāsādhyayanabrahmacaryavratāyuṣām ||6||
tadā bhūteṣvanukrośaṃ puraskṛtya maharṣayaḥ| sametāḥ puṇyakarmāṇaḥ pārśve himavataḥ śubhe||7||
aṅgirā jamadagniśca vasiṣṭhaḥ kaśyapo bhṛguḥ| ātreyo gautamaḥ sāṅkhyaḥ pulastyo nārado'sitaḥ||8||
agastyo vāmadevaśca mārkaṇḍeyāśvalāyanau| pārikṣirbhikṣurātreyo bharadvājaḥ kapiñja(ṣṭha)laḥ||9||
```

Fig. 2. A few lines of input from *Charaka Samhita* in the IAST schema.

```
dīrgham jīvitam-anvicchan-bharadvājaḥ upāgamat| indram-ugra-tapāḥ buddhvā śaraṇyam-amareśvaram||3||
brahmaṇā hi yathāproktam-āyurvedam prajāpatiḥ| jagrāha nikhilena-ādau=-aśvinau tu punar-tatas-||4||
aśvibhyām bhagavān-śakraḥ pratipede ha kevalam| ṛṣi-proktaḥ bharadvājaḥ-tasmāt-śakram-upāgamat||5||
vighna-bhūtāḥ yadā rogāḥ prādurbhūtāḥ śarīriṇām| tapa-upavāsa-adhyayana-brahmacarya-vrata-āyuṣām ||6||
tadā bhūteṣu-anukrośam puraskṛtya mahā-ṛṣayaḥ| sametāḥ puṇya-karmāṇaḥ pārśve himavataḥ śubhe||7||
aṅgirāḥ jamadagniḥ-ca vasiṣṭhaḥ kaśyapaḥ bhṛguḥ| ātreyaḥ gautamaḥ sāṅkhyaḥ pulastyaḥ nāradaḥ-asitaḥ||8||
agastyaḥ vāmadevaḥ-ca mārkaṇḍeya-aśvalāyanau| pārikṣiḥ-bhikṣuḥ-ātreyaḥ bharadvājaḥ kapiñja(ṣṭha)laḥ||9||
```

Fig. 3. Corresponding outputs of lines from *Charaka Samhita* given in Fig. 2. using Model A.

Ayurvedic classical texts. Another possibility is to develop a python-based tool/package which could be used to apply these *sandhi* splitting techniques to data easily. This could be helpful for carrying out the preprocessing stages while processing Sanskrit texts.

VI. CONCLUSION

This paper focused on a detailed study of the existing frameworks to solve the task of word segmentation and *sandhi* resolution for the Sanskrit language. From the reviewed models, some models with available resources were chosen to be used for further research in the project in the direction of Ayurvedic texts. A step was taken into introducing NLP tasks in the science of Ayurveda by compiling an Ayurveda corpus comprising the great classical scriptures of Ayurveda. Further, using this dataset, one of the chosen models was tested and the other models' testing is being carried out. Alongside this, the testing evaluation is also required to be carried out with human expert assistance in this field since no other source of data is available as the ground truth.

As future work, the compiled Ayurvedic dataset could be expanded by adding other Ayurvedic classical scriptures. Also, the word segmentation models could be improved by adding the Ayurveda domain-specific terminologies. Further, a word-embedding-based segmenter for the task has been planned to take this work to a new dimension.

REFERENCES

[1] Hellwig, Oliver. "Using Recurrent Neural Networks for joint compound splitting and Sandhi resolution in Sanskrit." In 4th Biennial workshop on less-resourced languages. 2015.

[2] Prasad, Vidya, B. Premjith, Chandni Chandran, K. P. Soman, and Prabaharan Poornachandran. "Deep learning based Character-level approach for Morphological Inflection Generation." In 2019 International Conference on Intelligent Computing and Control Systems (ICCS), pp. 1423-1427. IEEE, 2019.

[3] Narayanaswamy, V. "Origin and development of ayurveda:(a brief history)." Ancient science of life 1, no. 1 (1981): 1.

[4] Raj, Sreena, S. Karthikeyan, and K. M. Gothandam. "Ayurveda-A glance." Research in Plant Biology 1, no. 1 (2011).

[5] Reddy, Vikas, Amrith Krishna, Vishnu Dutt Sharma, Prateek Gupta, M. R. Vineeth, and Pawan Goyal. "Building a Word Segmenter for Sanskrit Overnight." In Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018). 2018.

[6] Nair, Jayashree, Sooraj S. Nair, and U. Abhishek. "Sanskrit stemmer design: A literature perspective." In International Conference on Innovative Computing and Communications: Proceedings of ICICC 2021, Volume 3, pp. 117-128. Springer Singapore, 2022.

[7] Krishna, Amrith, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. "A graph-based framework for structured prediction tasks in Sanskrit." Computational Linguistics 46, no. 4 (2021): 785-845.

[8] Premjith, B., K. P. Soman, and M. Anand Kumar. "A deep learning approach for Malayalam morphological analysis at character level." Procedia computer science 132 (2018): 47-54.

[9] Mittal, Vipul. "Automatic Sanskrit segmentizer using finite state transducers." In Proceedings of the ACL 2010 Student Research Workshop, pp. 85-90. 2010.

[10] Nair, Jayashree. "Generating noun declension-case markers for english to indian languages in declension rule based mt systems." In 2018 IEEE 8th International Advance Computing Conference (IACC), pp. 296-302. IEEE, 2018.

[11] Aralikatte, Rahul, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Mani. "Sanskrit Sandhi splitting using seq2 (seq)Ĵ2." arXiv preprint arXiv:1801.00428 (2018).

[12] Krishna, Amrith, Bishal Santra, Pavankumar Satuluri, Sasi Prasanth Bandaru, Bhumi Faldu, Yajuvendra Singh, and Pawan Goyal. "Word segmentation in sanskrit using path constrained random walks." In Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers, pp. 494-504. 2016.

[13] Hellwig, Oliver. "SanskritTagger: A Stochastic Lexical and POS Tagger for Sanskrit." Sanskrit Computational Linguistics 1 (2008): 266277.

[14] Goyal, Pawan, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. "A distributed platform for Sanskrit processing." In Proceedings of COLING 2012, pp. 1011-1028. 2012.

[15] Goyal, Pawan, and Gérard Huet. "Design and analysis of a lean interface for sanskrit corpus annotation." Journal of Language Modelling 4, no. 2 (2016): 145-182.

[16] Goyal, Pawan, and Gérard Huet. "Completeness analysis of a Sanskrit reader." In Proceedings, 5th International Symposium on Sanskrit Computational Linguistics. DK Printworld (P) Ltd, pp. 130-171. 2013.

[17] Hellwig, Oliver, and Sebastian Nehrdich. "Sanskrit word segmentation using character-level recurrent and convolutional neural networks." In Proceedings of the 2018 conference on empirical methods in natural language processing, pp. 2754-2763. 2018.

[18] Bhardwaj, Shubham, Neelamadhav Gantayat, Nikhil Chaturvedi, Rahul Garg, and Sumeet Agarwal. "Sandhikosh: A benchmark corpus for evaluating sanskrit sandhi tools." In Proceedings of the Eleventh In-

ternational Conference on Language Resources and Evaluation (LREC 2018). 2018.

[19] Chen, Xinchi, Xipeng Qiu, Chenxi Zhu, and Xuan-Jing Huang. "Gated recursive neural network for Chinese word segmentation." In Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pp. 1744-1753. 2015.

[20] Chen, Xinchi, Xipeng Qiu, Chenxi Zhu, Pengfei Liu, and Xuan-Jing Huang. "Long short-term memory neural networks for chinese word segmentation." In Proceedings of the 2015 conference on empirical methods in natural language processing, pp. 1197-1206. 2015.

[21] Premjith, B., Chandni Chandran, Shriganesh Bhat, Soman Kp, and P. Prabaharan. "A machine learning approach for identifying compound words from a Sanskrit text." In Proceedings of the 6th International Sanskrit Computational Linguistics Symposium, pp. 45-51. 2019.

[22] Li, Jingwen, and Leander Girrbach. "Word segmentation and morphological parsing for sanskrit." arXiv preprint arXiv:2201.12833 (2022).

[23] Bishop, Christopher M. Neural networks for pattern recognition. Oxford university press, 1995.

[24] Dave, Sushant, Arun Kumar Singh, Dr Prathosh AP, and Prof Brejesh Lall. "Neural compound-word (Sandhi) generation and splitting in Sanskrit language." In Proceedings of the 3rd ACM India Joint International Conference on Data Science Management of Data (8th ACM IKDD CODS 26th COMAD), pp. 171-177. 2021.

[25] Wu, Yonghui, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun et al. "Google's neural machine translation system: Bridging the gap between human and machine translation." arXiv preprint arXiv:1609.08144 (2016).

[26] Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. "Sequence to sequence learning with neural networks." Advances in neural information processing systems 27 (2014).

[27] Nair, Jayashree, and Anand Sadasivan. "A roman to devanagari back-transliteration algorithm based on Harvard-Kyoto convention." In 2019 IEEE 5th International Conference for Convergence in Technology (I2CT), pp. 1-6. IEEE, 2019.